Narrative Semantic Overlays as a Consistency Oracle for LLM Code Review

Status: completed pilot. Pre-registered design, executed; results in §5. Corpus: 30 files (15 defect-seeded, 15 bug-free twins). 4 conditions × 3 samples = 360 review instances. Model: `deepseek-chat`, temperature 1.0.

Abstract

Type systems and units-of-measure checkers catch a class of defect without reasoning about program purpose: they reject `Meters + Seconds` because the *sorts* disagree. We ask whether an LLM can be induced to perform an analogous check on code with no such type system, by assigning each variable a role in a small ontology — fictional ("this is a duck, that is a lion") or real (units, entity types) — together with rules constraining how roles may interact, and then flagging operations that map onto forbidden interactions.

We pre-registered a design that separates three confounded explanations for any benefit: that the *fiction* helps, that *structured relabeling* helps regardless of fiction, or that any *deliberation at all* helps. The critical comparison is against a deliberation-matched control, not a control that asks for an immediate answer.

The overlay does not improve detection. Against a deliberation-matched control, neither the fictional overlay (majority-of-3 detection 0.867 vs. 0.800) nor the real-domain overlay (0.800 vs. 0.800) differs significantly (McNemar, p = 1.0 for both; n = 15 paired files). Almost the entire apparent benefit of overlays over a naive control is reproduced by simply instructing the model to think first.

The overlay does change false-alarm behaviour, and that is the real finding. On bug-free twins, the naive control flagged all 45 review instances (3.31 flags/review; all 15 files were majority-flagged). Deliberation alone cut majority-flagged files to 7/15 (47%; 0.73 flags/review). The real-domain overlay cut this to 1/15 (7%; 0.16 flags/review), significantly below the deliberation-matched control (p = 0.031). The fictional overlay did *not* (9/15 files, 60%; 1.02 flags/review; p = 0.63 vs. Control+) and was significantly noisier than the real-domain overlay (p = 0.008).

So the arbitrariness claim — that a fictional ontology works as well as a real one — is **not supported**: the two arms detect equally, but only the real-domain ontology suppresses false alarms. The mechanism that pays is domain semantics, which is precisely what existing annotation-free unit analyzers already extract by static analysis, for free.

1. Introduction

A recurring class of defect is invisible to the compiler but obvious once named: arguments passed in the wrong order, a variable used where its similarly-named neighbour was meant, a

duration compared against a count. What unites them is that the *shape* of the computation is right and only the *identity* of the operands is wrong.

Type systems address a subset of this. Units-of-measure systems, dimensional analysis, and newtype/branded-type idioms all work by refusing to let semantically distinct quantities share a representation. Their power comes from the distinction being declared in advance and mechanically enforced.

Most code carries no such declarations. This paper asks whether an LLM reviewer can be prompted to *manufacture* the missing distinctions on the fly — invent an ontology post hoc, assign every variable a role in it, state rules over those roles, and check the code against rules it just authored.

1.1 The proposal decomposes into four claims

#	Claim	Status
C1	A role overlay helps a model *remember* code	Out of scope (§1.3)
C2	A role overlay helps a model *find bugs*	Tested; **not supported** (§5.1)
C3	The check is **context-free** — rules suffice, source not needed	**Not run** in this pilot (§7.1)
C4	The ontology may be **arbitrary and fictional** without loss	Tested; **not supported** (§5.2)

1.2 Hypotheses and outcomes

	Hypothesis	Outcome
H1	Overlay increases F1 on role-distinguishable defects, *relative to a deliberation-matched control*	**Not supported.** $\Delta F1 \leq 0.005$; McNemar $p = 1.0$
H2	Any effect is concentrated in swap / wrong-variable / unit classes	**Vacuous.** Those classes are at ceiling (0.89–1.00) in every condition, including control
H3	A checker seeing only roles + rules + symbolic trace detects above chance	**Not tested** (Treatment D deferred, §7.1)
H4	Fictional ≈ real-domain, both > scrambled	**Not supported.** Equal on detection; fictional significantly worse on false alarms ($p = 0.008$).

	Hypothesis	Outcome
		Scrambled arm not run

An unregistered result emerged and is reported as exploratory: the dominant effect of every intervention is on **false-positive suppression**, not detection (§5.3).

1.3 Scope: the memory claim is not tested

The originating idea began with *memory* — the mnemonic intuition that a narrative overlay makes material easier to hold in mind. A model performing a single-turn review has the source in context and never recalls it, so there is no recall step for an overlay to assist. Testing C1 needs a different design (present file, interpose distractor, require reconstruction with source withheld) and is left as future work rather than confounded in here.

2. Related work

2.1 Roles of variables

The closest prior work. A small fixed taxonomy of variable roles — fixed value, stepper, most-recent holder, gatherer, follower, walker, temporary, organiser, one-way flag, container — covers the overwhelming majority of variable occurrences in ordinary programs, and controlled studies report that teaching it improves *human* program comprehension relative to matched instruction without it [Sajaniemi 2002; Byckling & Sajaniemi 2006].

Three consequences. First, the general claim that role assignment aids reasoning about code is established for humans and is not novel. Second, there exists an off-the-shelf, empirically grounded role taxonomy — a better-motivated tight condition than invented `SORT_A/SORT_B` labels. Third, the taxonomy is neither fictional nor domain-specific, so it offers a principled midpoint for the C4 question.

2.2 Annotation-free unit-inconsistency detection

Static analyzers infer physical units *from variable names* and propagate them by dimensional analysis with no developer annotations. One reported system assigns units to roughly three quarters of variables at ~82% true-positive rate, and found inconsistencies in 11% of 213 open-source systems totalling over 900,000 lines at ~87% true-positive rate [Ore et al. 2017a]. Related systems extend this to reference frames [Ore et al. 2017b].

This is, in essence, Treatment C implemented as static analysis, predating the LLM framing. Treatment C is therefore the *incumbent*, not a weak control — and §5.2's finding that Treatment C is the only arm to buy anything makes this comparison decisive rather than academic (§6.2).

2.3 LLM type inference for untyped code

Directly relevant negative evidence. Benchmarks of LLM type inference over untyped Python report decent type *similarity* but significant type *consistency* errors across a codebase [Peng et al. 2023]. The overlay's entire mechanism is the self-consistency of a self-authored labeling, so

documented failures in exactly that operation predict H1's failure. Our between-sample instability figures (§5.4) are consistent with this.

2.4 Prompting strategies that add a reasoning pass

Chain-of-thought and related techniques improve performance on many tasks simply by inducing intermediate reasoning before an answer [Wei et al. 2022; Kojima et al. 2022]. This is the principal threat to any positive result here: a narrative overlay *is* such a representation, and any benefit might come from having one at all. The deliberation-matched Control+ condition exists to neutralize it — and §5.1 shows it absorbs essentially the entire effect.

2.5 Mutation testing and defect benchmarks

Corpus construction follows mutation-testing practice: apply a single, documented, localized change to a known-good program and treat its detection as the measurement. Defects4J [Just et al. 2014] and BugsInPy [Widyasari et al. 2020] are the standard curated benchmarks. We deliberately do not draw from them (§3.1), because a model may have memorized public benchmark defects.

2.6 Assessment of novelty

The specific combination — an *arbitrary, self-authored, fictional* ontology used as a consistency oracle by an LLM — does not appear to have been studied. The surrounding claims largely have been: role assignment aiding comprehension (2.1), name-derived unit checking (2.2), structured intermediate representations aiding reasoning (2.4). We therefore never presented C2 as the contribution. The results now retire C4 as well.

3. Corpus

3.1 Construction

Hand-written Python files of 25–40 lines spanning three domains — numeric / unit-bearing code, entity-ID handling, and pure control flow. Each has a defect-seeded twin. Pilot corpus: 15 file pairs = 30 files.

Files are hand-authored rather than drawn from Defects4J or BugsInPy specifically to avoid the memorization threat. The cost is that our defects are ours and may not reflect the natural distribution of real-world bugs.

Provenance caveat. The corpus was authored by a model. Model-written code may be unusually legible, and its naming conventions may align with an evaluator's priors — which would plausibly inflate the treatments, since they depend on inferring roles from names, the same signal §2.2's analyzers exploit. The pilot ran a single evaluator family; §7.2 records this.

3.2 Defect classes and full crossing

H2 predicts a class-dependent effect, so class must not be confounded with domain. The built corpus satisfies a complete crossing: each of the five classes draws its three files from all three domains.

Class	Domains	H2 prediction	Observed (§5.4)
Swapped operands	all 3	effect	ceiling, no effect
Wrong variable	all 3	effect	ceiling (1.00 everywhere)
Unit / type mismatch	all 3	effect	flat 0.667 everywhere
Off-by-one / control flow	all 3	null	no effect
Missing null / bounds check	all 3	null	noisiest cell, floor effects

3.3 Reproducible generation and divergence witnesses

The seeded corpus is generated, never hand-edited. A build script applies one exact string substitution per file and aborts if any pattern fails to match exactly once, so ground truth cannot drift from the files.

A mechanical **divergence check** executes the correct and seeded versions over randomized inputs and requires at least one observable divergence, aborting the build if none is found. This was added after an early hand-authored mutation — swapping the operands of `abs(destination_zone - origin_zone)` — turned out to be behaviour-preserving and therefore undetectable in principle.

4. Design

4.1 Conditions run

Condition	Description	Isolates	Run?
Control	"Review this code for bugs." Report only, no prose.	Floor	✅
Control+	Same task, instructed to reason in writing first. No roles or ontology.	Deliberation without relabeling	✅
Treatment A	Fictional bestiary + informal narrative rules	Fiction	✅
Treatment C	Real domain roles (units, entity types) + real compatibility rules	Real semantics; the incumbent	✅
Treatment A-	A with	Coherence vs.	⛔ deferred

Condition	Description	Isolates	Run?
scrambled	assignments permuted	ritual	
Treatment B	Opaque sorts + permitted-operation table	Structure without meaning	⛔ deferred
Treatment D	Two-stage decoupled check	Context-freeness (H3)	⛔ deferred

The pilot ran the four conditions that decide H1 and the detection half of H4. §7.1 states what the deferred arms would still settle.

4.2 The comparison ladder

Rung	Comparison	Question
1	Control vs. Control+	Does any deliberation help?
2	Control+ vs. Treatments A and C	Does relabeling add anything beyond deliberation? **Primary**
3	Treatment A vs. Treatment C	Does arbitrariness cost anything?

Rung 1 is diagnostic: if the Control→Control+ gap is large, any two-condition study's result was this effect wearing a costume. Rung 2 is H1 and the primary outcome. Rung 3 is the motivating question — if real-domain relabeling performs as well as the fictional overlay, the originating idea is unsupported even if A beats a naive baseline handily.

4.3 Procedure

Each (file, condition, sample) is an independent single-turn request with no conversation history, so an ontology built for one file cannot leak into the next. Source is presented with line numbers prepended, identically in every condition. All conditions demand identical output: `FLAG line=N :: description` or `NO BUGS FOUND`; reasoning conditions emit it after a `=== REPORT ===` marker.

Repeated sampling. k = 3 samples per (file, condition) at temperature 1.0. At k = 1 it is impossible to distinguish "the treatment does not work" from "the treatment works when the ontology lands well, and it lands well half the time." Per-file outcome is **majority-of-3** (pre-registered primary), with best-of-3 and unanimous-3 as sensitivity analyses.

Truncation. Every treatment places its report *last*, so a truncated response loses precisely the scoreable part and would be scored as a clean miss — an artifact acting on treatments only. The runner records stop reason; **all 360 instances terminated normally** (`stop`), none at the token cap, so truncation did not create false negatives.

One non-truncated Treatment A response (the buggy tenant-lookup file, sample

1) contained reasoning but no parseable `FLAG` or `NO BUGS FOUND` result. It was retained as a miss, not retried or excluded; the published metrics use the full planned denominator.

4.4 Measures

Precision, recall, and F1 over buggy review instances; false-positive rate on the bug-free twins, reported as both flagged review instances and majority-flagged files; flags and input/output tokens per review; and between-sample stability. Precision is computed over buggy review instances, with clean-file flags reported separately as the false-positive rate.

Flags per review is reported prominently because a shotgun reviewer buys recall cheaply and a ±k-line criterion covers a substantial fraction of a 30-line file.

4.5 Scoring and blinding

A flag counts as a true positive if it names the correct line **and** correctly characterizes the defect. Only the first is mechanical. The apparatus computes line proximity at **±1 line** (primary criterion, pre-registered; ±2 reported as sensitivity) and leaves characterization to blinded adjudication.

Detection results in §5 are line-match figures, which are an *upper bound* on adjudicated performance. The adjudication pass is outstanding (§7.3); flags landing on the right line for the wrong reason are counted as hits here.

Blinding. Transcripts are reduced to their `FLAG` lines with a vocabulary scrub over descriptions, removing overlay reasoning and condition-revealing terms. A **manipulation check** had an independent model identify the condition from blinded flags alone: **11/60 correct = 18.3% against 25% chance (p = 0.914)**. Blinding held — the checker performed no better than chance, and in fact numerically worse.

5. Results

All 360 review instances completed. Detection metrics use ±1-line matches; clean false-alarm metrics count every emitted flag.

5.1 H1: the overlay does not improve detection

Condition	Sample rate	Majority-of-3	Best-of-3	Unanimous-3
Control	0.756	0.733 (11/15)	0.800	0.733
Control+	0.822	**0.800 (12/15)**	0.867	0.800
Treatment A (fiction)	0.844	0.867 (13/15)	0.933	0.733
Treatment C (real domain)	0.756	0.800 (12/15)	0.867	0.600

Exact McNemar on paired per-file majority outcomes, against Control+:

Comparison	b	c	p
Control+ vs. Control	1	0	1.00
Control+ vs. Treatment A	1	0	1.00
Control+ vs. Treatment C	1	1	1.00

Every detection comparison is null. Treatment A's nominal +1 file over Control+ is a single discordant pair. Notably Treatment A's *unanimous-3* rate (0.733) falls **below** Control+'s (0.800), and Treatment C's falls further (0.600): the overlays win marginally on best-of-3 and lose on consistency, which is the signature of higher variance rather than higher skill.

H1 is not supported.

5.2 H4: fiction is not free

The two overlay arms detect identically. They differ sharply in false alarms.

Condition	Flagged clean review instances (of 45)	Files majority-flagged (of 15)	Flags per clean review	vs. Control+ (p)
Control	45 (100%)	15 (100%)	3.31	0.0078 (worse)
Control+	23 (51%)	7 (47%)	0.73	—
Treatment A (fiction)	28 (62%)	9 (60%)	1.02	0.63 (n.s.)
Treatment C (real domain)	**5 (11%)**	**1 (7%)**	**0.16**	**0.031 (better)**

Treatment A vs. Treatment C, McNemar on majority-flagged clean files: b = 8, c = 0, **p = 0.0078**. The fictional overlay is significantly noisier than the real-domain overlay while detecting no better.

H4 is not supported. Arbitrariness is not free: it costs the entire false-alarm benefit, which is the only benefit either overlay delivers.

This was predicted. Ten of fifteen bug-free files carry unit-bearing identifiers, and several are *ill-sorted but correct* — one computes latitudes in radians while deriving deltas from the corresponding degree variables (correct, since the conversion is linear), another legitimately holds Fahrenheit and Celsius simultaneously. A checker faithfully applying its own table should flag these. The real-domain ontology evidently recognizes the conversions as legitimate; the fictional one, having no semantics to appeal to, does not.

5.3 The dominant effect is precision, not recall

Condition	Precision	Recall	F1	Flags/ buggy review	Out tokens
Control	0.337	0.756	0.466	2.24	221
Control+	0.712	0.822	**0.763**	1.16	1183
Treatment A	0.704	0.844	**0.768**	1.20	1489
Treatment C	0.708	0.756	0.731	1.07	1080

Recall varies across a 0.09 band. Precision varies across a 0.38 band, and the whole of that variation sits between Control and *everything else*. The naive control is a shotgun: 2.24 flags per buggy review, 3.31 per clean review, and at least one flag in every clean review. Any instruction to deliberate collapses this.

The F1 ranking (A 0.768 ≈ Control+ 0.763 > C 0.731) is what a two-condition study comparing Treatment A against a naive control would have reported as a large win (0.768 vs. 0.466, +0.30 F1). **That entire gap is deliberation.** The overlay-specific increment over a deliberation-matched control is +0.005.

Cost: the overlays spend 4.9–6.7× the control's output tokens and 0.9–1.3× the Control+ baseline's. Treatment C is the cheapest reasoning condition (1080 tokens) and the most precise, which is the only favourable cost story in the table — and it is a story about domain semantics, not about narrative.

5.4 H2 and stability: class effects are masked by ceiling

Per-class detection (review-level, buggy files):

Class	Control	Control+	Treat. A	Treat. C
swapped_operands	1.000	1.000	1.000	0.889
wrong_variable	1.000	1.000	1.000	1.000
unit_type_mismatch	0.667	0.667	0.667	0.667
off_by_one	0.778	1.000	0.889	0.889
missing_check	0.333	0.444	0.667	0.333

H2 predicted an overlay effect concentrated in the first three rows. Swap and wrong-variable detection are high in every condition, while `unit_type_mismatch` is pinned at 0.667 in all four arms. The class cells have only three files each, so apparent differences elsewhere (including `missing_check`) are exploratory and not interpretable. **H2 is untestable on this corpus**, not refuted.

Between-sample instability (files where the three samples disagree):

Condition	Unstable files
Control	1/15
Control+	1/15
Treatment A	3/15
Treatment C	4/15

The overlay conditions are 3–4× less stable across samples than either control. This is the §2.3 prediction realized: the overlay's power depends on where the model draws its sort boundaries, and it draws them differently each time. It also explains the best-of-3 / unanimous-3 inversion in §5.1 — the overlays sometimes land a favourable ontology and sometimes do not.

6. Discussion

6.1 What the deliberation-matched control bought

The single most consequential design decision was adding Control+. Without it, this experiment would have reported a +0.30 F1 improvement for narrative overlays, a 37-point precision gain, and a clean H1 confirmation. All of it is chain-of-thought. This is a concrete instance of a general hazard: any prompt that induces structured intermediate reasoning must be compared against unstructured intermediate reasoning, not against no reasoning at all.

6.2 The one real effect points away from the hypothesis

Treatment C's false-alarm rate — 5/45 clean review instances flagged, with 1/15 files majority-flagged, against Control+'s 23/45 instances and 7/15 files — is the only significant treatment effect against Control+ in the study. It is produced by the arm the design nominated as *the incumbent*: real domain roles, real units, real compatibility rules. That is what §2.2's static analyzers already extract from variable names, at zero marginal cost per invocation and with none of the sampling instability in §5.4.

The practical reading is unflattering to the proposal. If you want fewer false alarms on correct code, ask the model to reason about units and entity types — or run a unit-inconsistency analyzer. Wrapping that in a bestiary does not help, and measurably hurts.

6.3 Why fiction underperforms

The fictional overlay must invent both the sorts and the rules. When correct code is *ill-sorted but semantically fine* — degrees alongside radians, Fahrenheit alongside Celsius — a real-domain checker has an appeal available ("these are the same quantity in different units, the conversion is linear") that an arbitrary ontology does not. A duck cannot become a lion. The very arbitrariness that made the proposal appealing — no domain knowledge required — is what removes the escape hatch that prevents false alarms.

7. Limitations

7.1 Three arms were not run

Treatment D (context-freeness, H3) is the sharpest untested claim. The originating proposal asserts the check needs only rules and assignments, not source. The pilot never put that at risk. Its two-stage form — a builder emitting only a role table, a rule set, and a symbolic trace (`L33: DUCK * RIVER -> ?`); a fresh cheaper checker seeing only that artifact — remains informative in both directions. If Stage 2 succeeds, the strong claim is vindicated and yields a practical pattern. If it collapses, the overlay is a reasoning scaffold rather than an oracle. Given §5.4's instability figures, the latter is the prediction.

Treatment A-scrambled would settle whether coherence matters or any labeling ritual suffices — the most deflationary available outcome, and cheap. **Treatment B** (opaque sorts + explicit permitted-operation table) would separate structure from meaning, which §6.3 currently explains only by inference.

7.2 Single model, single family

All 360 instances ran on `deepseek-chat`. The corpus was model-authored (§3.1), raising a legibility concern that a second evaluator family would address. Nothing here establishes that these effects generalize across models.

7.3 Line-match only; adjudication outstanding

All reported figures are ±1-line matches, an **upper bound**. The `true_positive_adjudicated` column is unpopulated. A flag landing on the right line while describing a different problem is counted as a hit here and would be scored wrong under the pre-registered rubric. The direction of this bias is plausibly against the overlays — their flag descriptions are ontology-framed and more likely to mischaracterize — which would strengthen, not weaken, the null conclusions.

Inter-rater reliability (Cohen's κ on a 20% double-adjudicated subsample, pre-registered floor $\kappa \geq 0.7$) is likewise outstanding.

7.4 Power, ceiling, and corpus scale

n = 15 paired files gives McNemar very little power: with 15 pairs a 1-vs-0 discordant split cannot reach significance under any circumstances. The null detection results should be read as "no effect large enough to see at this scale," not "no effect." The class cells have only three files each, so even the apparent differences in §5.4 cannot support reliable class-level inference. A larger corpus with harder seeded defects is a precondition for testing H2.

The clean-file contrasts reached nominal significance despite the small sample: the paired comparison of Treatment C with Control+ had 6-vs-0 discordant files, and Treatment C with Treatment A had 8-vs-0. There are only 15 independent files, so these results still warrant replication.

7.5 No non-LLM baseline

For the unit-mismatch class we intended to run an existing annotation-free unit-inconsistency analyzer over the same files. This was not done. Given §6.2, it is now the most important missing comparison in the study: Treatment C's only real effect is in exactly the territory those tools already cover.

8. Conclusion

A narrative semantic overlay does not make an LLM a better bug finder. Against a deliberation-matched control, it detects no more defects ($p = 1.0$), is 3–4× less stable across samples, and costs more tokens. The large improvement it shows against a naive control is chain-of-thought in a costume.

The one significant treatment effect runs against the proposal's central claim. Real-domain role labeling cut flagged clean review instances from 23/45 (51%) to 5/45 (11%), or majority-flagged files from 7/15 (47%) to 1/15 (7%; $p = 0.031$). The fictional overlay did not reduce false alarms relative to Control+ (28/45 reviews; 9/15 files; $p = 0.63$) and was noisier than the real-domain arm ($p = 0.008$). Fiction is not free. What pays is domain semantics — which annotation-free static analyzers already infer from variable names at no per-invocation cost.

The sharpest claim, that the check is context-free, remains untested and is the one experiment still worth running.

Appendix A — Reproduction

```
harness/build_corpus.py    generate seeded twins; abort on ambiguous
match
harness/divergence.py      execute both twins; require observable
divergence
harness/run_pilot.py       360 single-turn instances, k=3, temp 1.0
harness/blind.py           reduce to FLAG lines + vocabulary scrub
harness/guess_condition.py manipulation check (18.3% vs 25% chance)
harness/score.py           ±1 line match primary, ±2 sensitivity
harness/analyze.py         McNemar, Holm-Bonferroni, per-class
breakdown
```

Artifacts: `results/results.csv` (360 rows), `results/transcripts.jsonl`, `results/blinded.jsonl`, `results/blinding_key.json`, `results/manipulation_check.json`, `results/flags_for_adjudication.csv` (pending §7.3).

Appendix B — References

- Sajaniemi, J. (2002). An empirical analysis of roles of variables in novice-level programming courses. *IEEE HCC*. https://doi.org/10.1109/HCC.2002.1046334
- Byckling, P. & Sajaniemi, J. (2006). Roles of variables and programming skills improvement. *SIGCSE*. https://doi.org/10.1145/1121341.1121428
- Ore, J.-P., Detweiler, C. & Elbaum, S. (2017a). Lightweight detection of physical unit inconsistencies without program annotations. *ISSTA*. https://doi.org/10.1145/3092703.3092722

- Ore, J.-P., Elbaum, S. & Detweiler, C. (2017b). Dimensional inconsistencies in code and ROS messages. *IROS*. https://doi.org/10.1109/IROS.2017.8206007
- Peng, Y. et al. (2023). Generative type inference for Python. *ASE*. https://doi.org/10.1109/ASE56229.2023.00031
- Wei, J. et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*. https://arxiv.org/abs/2201.11903
- Kojima, T. et al. (2022). Large language models are zero-shot reasoners. *NeurIPS*. https://arxiv.org/abs/2205.11916
- Just, R., Jalali, D. & Ernst, M. D. (2014). Defects4J: a database of existing faults to enable controlled testing studies for Java programs. *ISSTA*. https://doi.org/10.1145/2610384.2628055
- Widyasari, R. et al. (2020). BugsInPy: a database of existing bugs in Python programs. *ESEC/FSE*. https://doi.org/10.1145/3368089.3417943